Build a Q&A API with Python & Django

While you wait, run these commands in a terminal:

```
git clone https://github.com/joshualent/questions-api.git
cd questions-api
uv sync --locked
uv run manage.py migrate
uv run manage.py runserver
```

Then open `http://127.0.0.1:8000/admin/`. You should see the Django login page.

If you don't have Python 3.14, `uv sync` installs it.



github.com/
joshualent/
questions-api

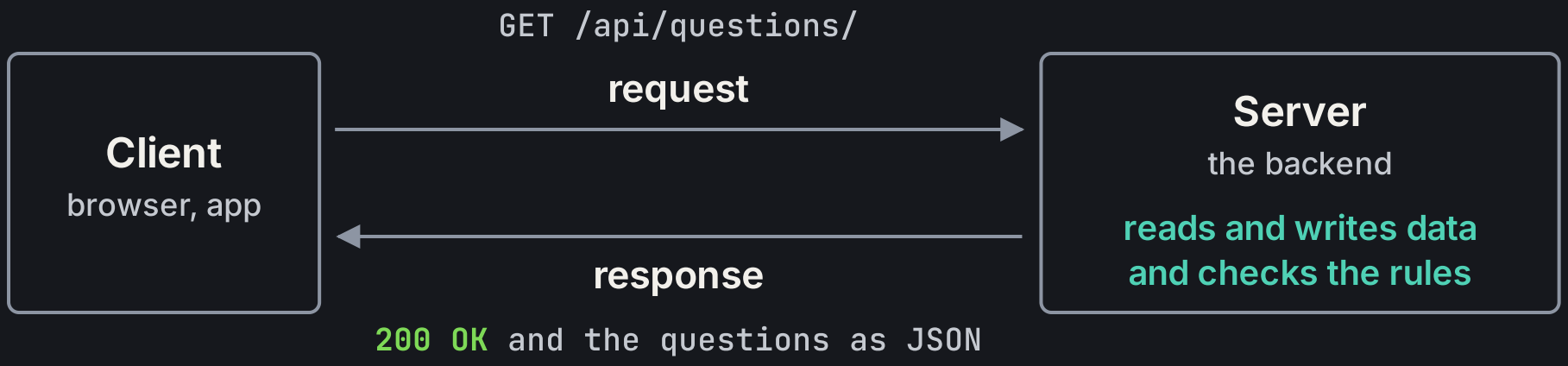
Placeholder: Developer Club logo (set logo in workshop.config.ts)

Build a Q&A API with Python & Django

Backend workshop · Developer Club, WMU

Monday, October 19, 2026 · Floyd Hall D-109

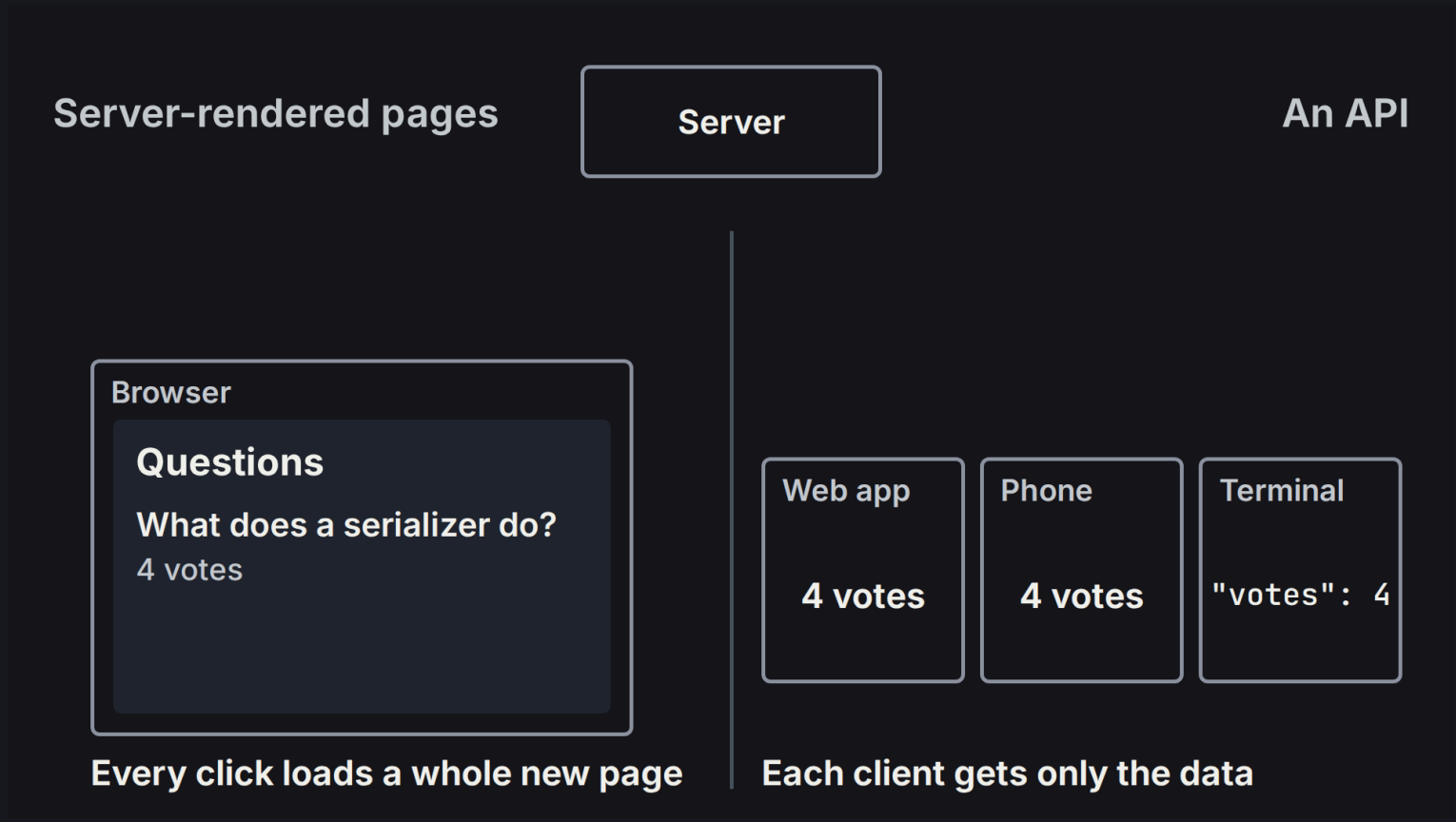
What a backend does



What the backend is responsible for

- It stores the data: questions, votes, and users.
- It enforces rules. For example, a question can't be blank.
- It decides who can do what. For example, only moderators can delete questions.

Server-rendered pages vs. an API



Why build an API?

- One backend can serve many clients, such as a web app, a phone app, and a terminal.
- You can change and deploy the frontend without changing the backend.
- You can test and scale the frontend and the backend separately.

Anatomy of a request

Request

```
POST /api/questions/ HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"text": "What does a serializer do?"}
```

Response

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id": 1,
 "text": "What does a serializer do?",
 "votes": 0,
 "answered": false,
 "created_at": "…"}
```

The server filled in id, votes, answered, and created_at.

The methods and status codes we use tonight

Methods

`GET`	read
`POST`	create, or run an action
`PUT` `PATCH`	edit
`DELETE`	delete

Status codes

`200 OK`	success
`201 Created`	created
`400 Bad Request`	invalid data, such as blank text
`403 Forbidden`	you aren't allowed to do this
`404 Not Found`	the question doesn't exist

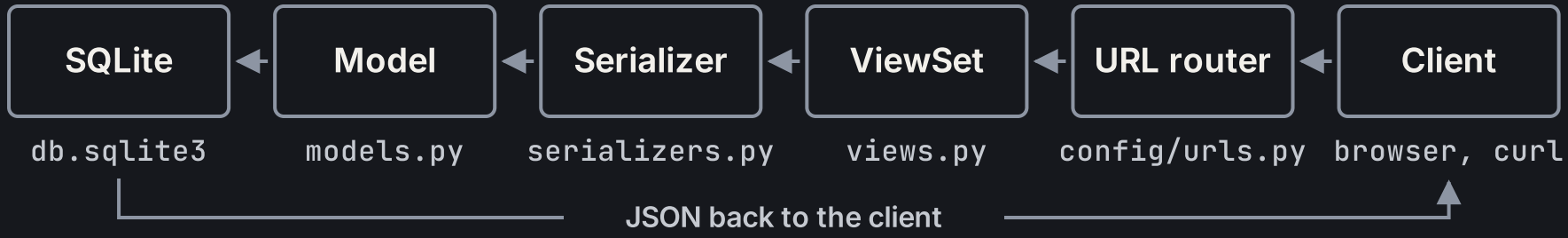
Resources and URLs

`/api/questions/` is the collection of all questions.
`/api/questions/1/` is the question with id 1.

`GET`	`/api/questions/`	list all questions
`POST`	`/api/questions/`	create a question
`GET`	`/api/questions/1/`	read question 1
`PUT`	`/api/questions/1/`	replace question 1
`PATCH`	`/api/questions/1/`	change part of question 1
`DELETE`	`/api/questions/1/`	delete question 1

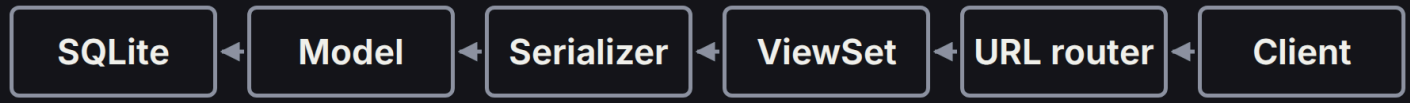
In lesson 2 we add actions, such as `POST` `/api/questions/1/upvote/`

The parts of our API



We build these parts from left to right, starting with the model.

Life of a POST request



The request and the response

Sent `{"text": "What does a serializer do?", "votes": 9999}`

Got back `201 Created` `{"id": 1, …, "votes": 0, …}`

Blank text `400 Bad Request` `{"text": ["This field may not be blank."]}`

The client asked for 9999. The server stored 0.

Your starter repo

```
questions-api/
  config/
    settings.py
    urls.py
  questions/
    admin.py
    models.py
    serializers.py
    views.py
  manage.py
  pyproject.toml
  README.md
```

Checkpoint branches

`checkpoint-0-setup`	where you are now
`checkpoint-1-crud`	after lesson 1
`checkpoint-2-actions`	after lesson 2
`checkpoint-3-auth`	after lesson 3
`checkpoint-4-m2m`	the finished API

If you fall behind, run:

```
git stash push -u -m "my work"
git switch checkpoint-1-crud
uv run manage.py migrate
```

Lesson 1: CRUD

Store questions in a database, and create, read, update, and delete them over HTTP.

A model becomes a database table

Each model becomes a table, and each field becomes a column.

	models.py		questions_question	
text	CharField	→	varchar(300)	at most 300 characters
votes	IntegerField	→	integer	defaults to 0
answered	BooleanField	→	bool	defaults to false
created_at	DateTimeField	→	datetime	set when it's created
id	BigAutoField	→	integer	Django adds it: 1, 2, 3, ...

`makemigrations` writes a migration file. `migrate` applies it.



A serializer converts and validates data

A serializer converts JSON to Python objects and back. It also validates incoming data before anything is saved.

`POST` `/api/questions/` `{"text": "hi", "votes": 9999}`
`201 Created` `{"id": 2, "text": "hi", "votes": 0, …}`
votes is read-only, so the serializer ignores it.

`POST` `/api/questions/` `{"text": ""}`
`400 Bad Request` `{"text": ["This field may not be blank."]}`
Blank text fails validation, so the server saves nothing.

Always validate data that comes from the client.



One ViewSet handles six routes

A ViewSet is one class that handles every operation on a resource. Ours is `QuestionViewSet`.

DRF's `DefaultRouter` creates six routes from it:

GET	/api/questions/	list
POST	/api/questions/	create
GET	/api/questions/1/	retrieve
PUT	/api/questions/1/	update
PATCH	/api/questions/1/	partial_update
DELETE	/api/questions/1/	destroy

For now, `AllowAny` lets anyone call every route. Lesson 3 adds restrictions.



What’s still broken?

Works	Clients can create, list, read, edit, and delete questions.
Doesn't work	Clients can't change `votes` or `answered`, because both fields are read-only.

How should voting work?

Why not let clients PATCH the vote count?

`PATCH` `/api/questions/1/` `{"votes": 9999}`

If votes were writable, any client could set any number.

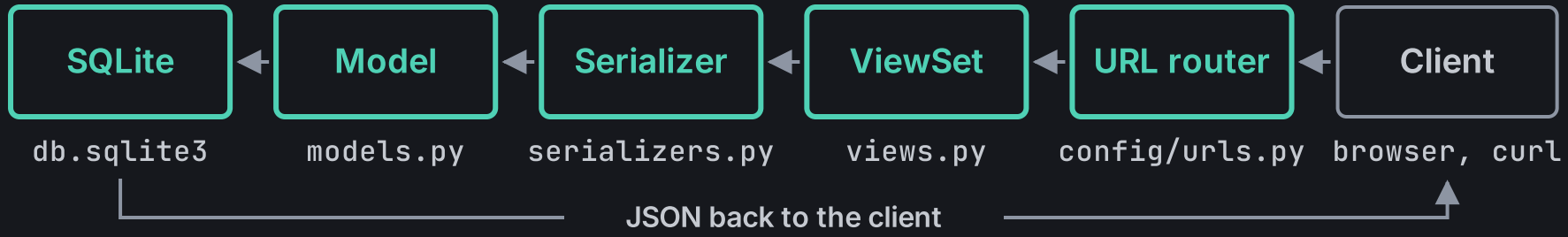
`POST` `/api/questions/1/upvote/` no body

The server applies the rule: each upvote adds one.

Some operations aren't create, read, update, or delete. DRF calls them **actions**.

`detail=True` makes the action work on one question, so its URL includes the question's id.

You built a REST API



If you're behind, catch up to the end of lesson 2:

```
git stash push -u -m "my work"
git switch checkpoint-2-actions
uv run manage.py migrate
```

What’s still broken?

- Anyone can upvote a question as many times as they want, without logging in.
- Anyone can mark a question answered.
- Anyone can edit or delete any question.

Authentication and authorization

Authentication checks who sent the request. You prove who you are with a username and password.

Authorization checks whether that user is allowed to make the request. In DRF, permission classes do this.

Tonight we log in to the browsable API with a session. Apps and frontends usually send a token instead. The permission code is the same for both.

Who can do what

Operation	Who can do it
List, read, or create questions	Anyone, including logged-out visitors
Upvote	Any logged-in user
Edit or delete questions	Staff users
Toggle answered	Staff users

Staff status is a checkbox on each user. DRF's `IsAdminUser` checks staff status, not superuser status. `createsuperuser` sets both.

How DRF checks permissions

A request reaches `QuestionViewSet`

Is it update, partial_update, or destroy?
`get_permissions()`

yes → `IsAdminUser` staff only

no

Does the @action set permission_classes?
`@action(permission_classes=…)`

yes → `upvote` `IsAuthenticated`
`toggle_answered` `IsAdminUser`

no

Every other action: list, create, retrieve
`permission_classes = [AllowAny]`

→ `AllowAny` anyone



What's still broken?

`sam` upvotes three times:

```
POST  /api/questions/1/upvote/
200 OK  "votes": 1

POST  /api/questions/1/upvote/
200 OK  "votes": 2

POST  /api/questions/1/upvote/
200 OK  "votes": 3
```

The votes field stores only a number, so the server can't tell that sam already voted.

Lesson 4 stores which users voted for each question, instead of a number.

Keep the JSON compatible

Clients see only the JSON. If the existing fields keep their names and types, existing clients keep working.

Before: `votes` is an integer column

```
{"id": 1,
 "text": "What does a serializer do?",
 "votes": 1,
 "answered": false, …}
```

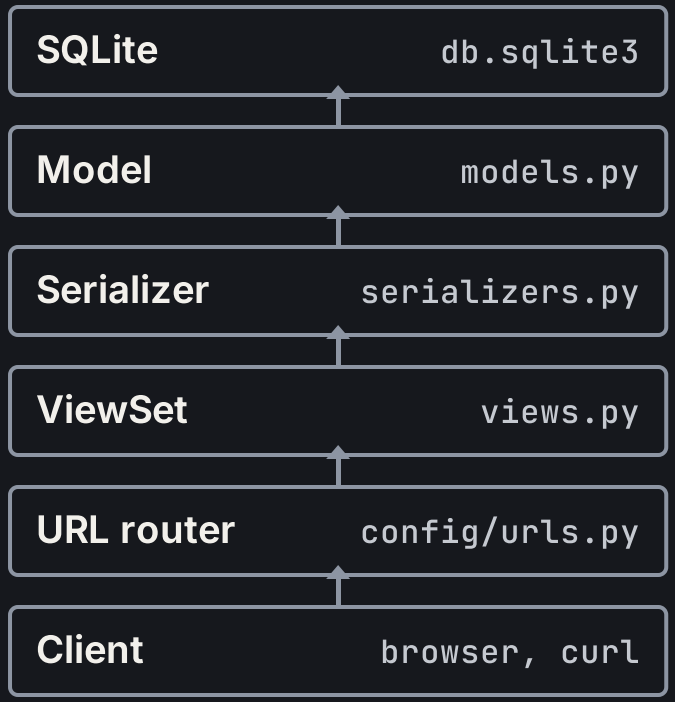
After: `votes` counts the users who voted

```
{"id": 1,
 "text": "What does a serializer do?",
 "votes": 1,
 "has_voted": false,
 "answered": false, …}
```

The serializer reports `votes` as `votes.count`, so `votes` is still a number. It also adds a new field, `has_voted`.

Renaming `upvote` to `toggle_vote` changes the URL. Clients that still call `/upvote/` get a 404.

What you built



`questions_question` `questions_question_votes`

`Question`

`QuestionSerializer`

`QuestionViewSet` `get_permissions` `toggle_vote`

`DefaultRouter`

Where to go next

- The official DRF tutorial
- Tests with DRF's `APITestCase`
- Pagination and ordering
- A frontend that calls your API
- Deploying



Questions?

Thanks for coming. The finished code is on `checkpoint-4-m2m`.

Next Developer Club event

Placeholder: next event (set nextEvent in workshop.config.ts)

Placeholder: Developer Club logo (set logo in workshop.config.ts)



github.com/
joshualent/
questions-api

Appendix

30 Switching checkpoints

31 Troubleshooting

32 Video still: one backend, many clients

33 Video still: life of a POST request

34 Projector check

Switching checkpoints

Save your work, switch branches, and update the database:

```
git stash push -u -m "my work"
git switch checkpoint-3-auth
uv run manage.py migrate
```

To go back from checkpoint-4-m2m, undo its migration before you switch:

```
uv run manage.py migrate questions 0001
git switch checkpoint-3-auth
```

To start fresh on any checkpoint, stop the server, delete `db.sqlite3`, and run `migrate`.

Troubleshooting

That port is already in use

```
uv run manage.py runserver 8001
```

no such table

```
uv run manage.py migrate
```

uv: command not found (macOS, Linux)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

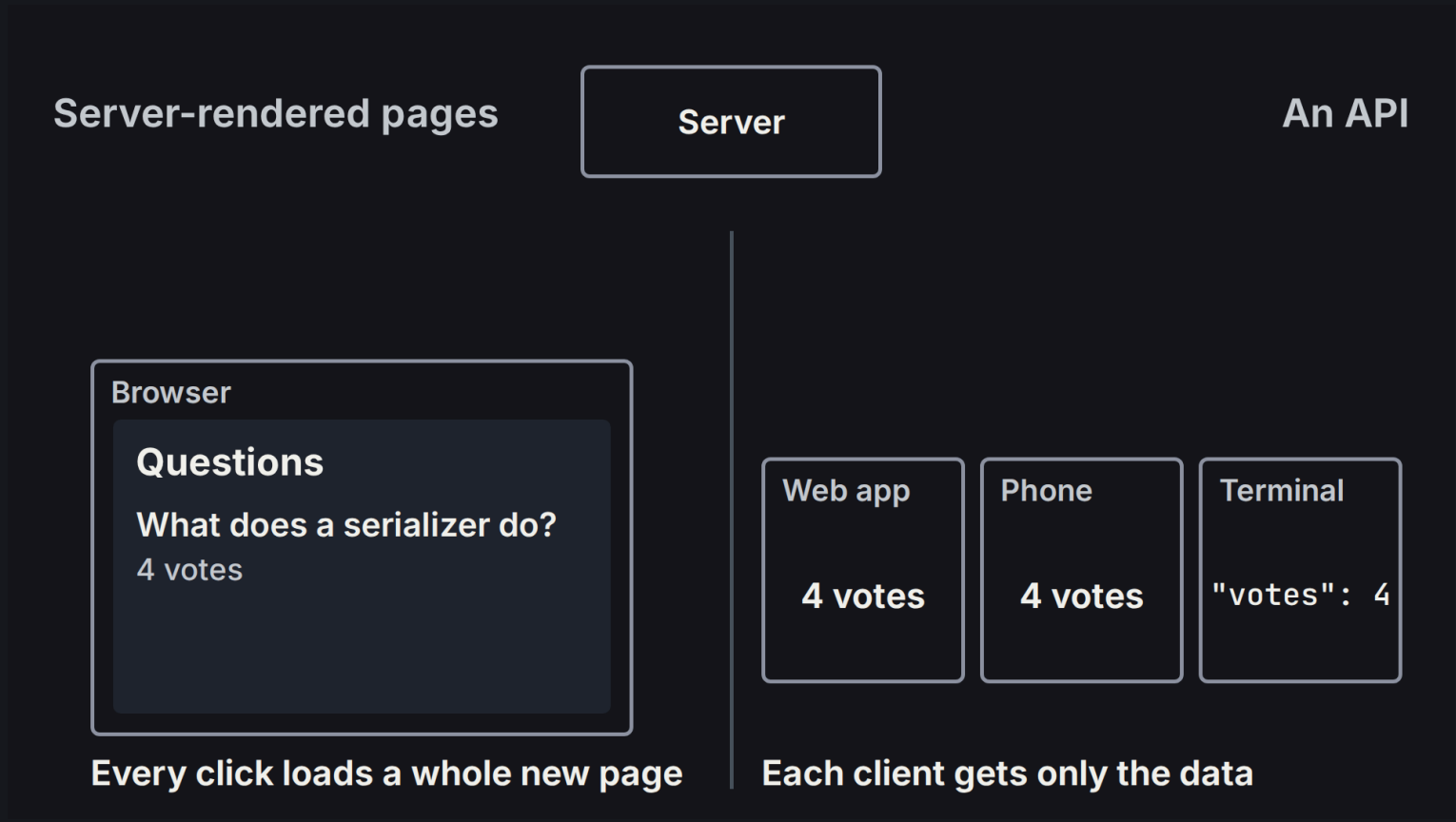
uv: command not found (Windows PowerShell)

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

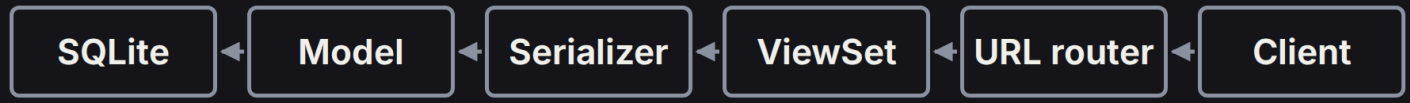
No module named django

```
use uv run manage.py …, not python manage.py …
```

Video still: one backend, many clients



Video still: life of a POST request



The request and the response

Sent `{"text": "What does a serializer do?", "votes": 9999}`

Got back `201 Created` `{"id": 1, …, "votes": 0, …}`

Blank text `400 Bad Request` `{"text": ["This field may not be blank."]}`

The client asked for 9999. The server stored 0.

Projector check

- background
- code surface
- text
- secondary text
- diagram lines
- table rules
- accent
- GET
- POST
- PUT / PATCH
- DELETE
- 2xx
- 4xx

Body text, 24 px: the smallest body size in the deck.

Secondary text, 24 px: still readable at the back?

```
Code, 18 px: question.votes.add(request.user)
```

Diagram label, 16 px: config/urls.py · serializers.py · db.sqlite3

GET POST PATCH DELETE 201 Created 403 Forbidden

```
class Question(models.Model):
    text = models.CharField(max_length=300)
    votes = models.IntegerField(default=0)
```